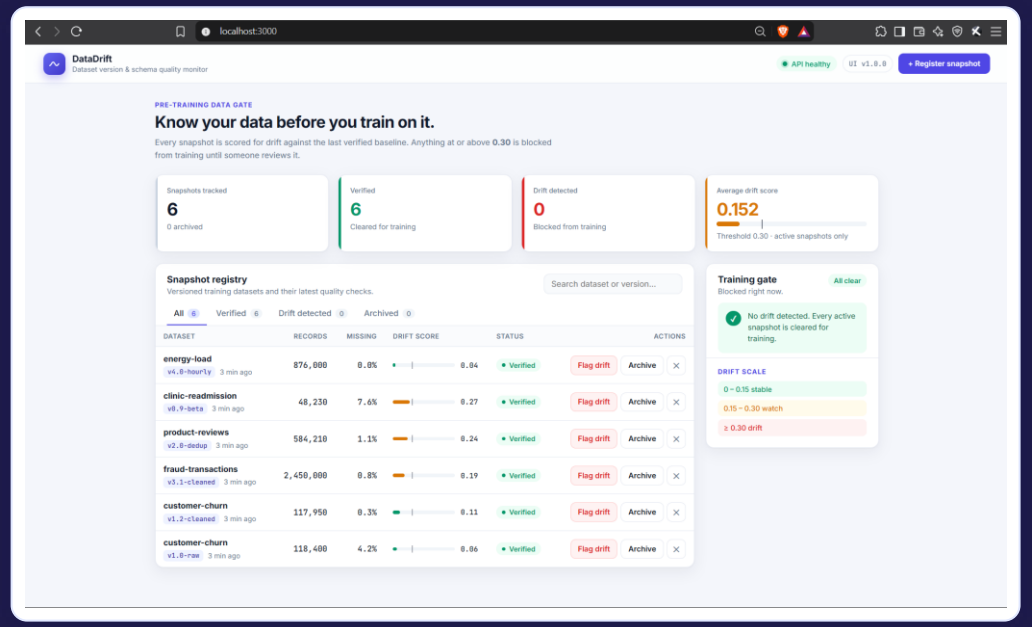
DEVOPS CAPSTONE PROJECT

DataDrift

Dataset version & schema quality monitor



Kavya Dhyani
github.com/KavyaDhyani/datadrift-capstone

Know your data before you train on it

Register every snapshot

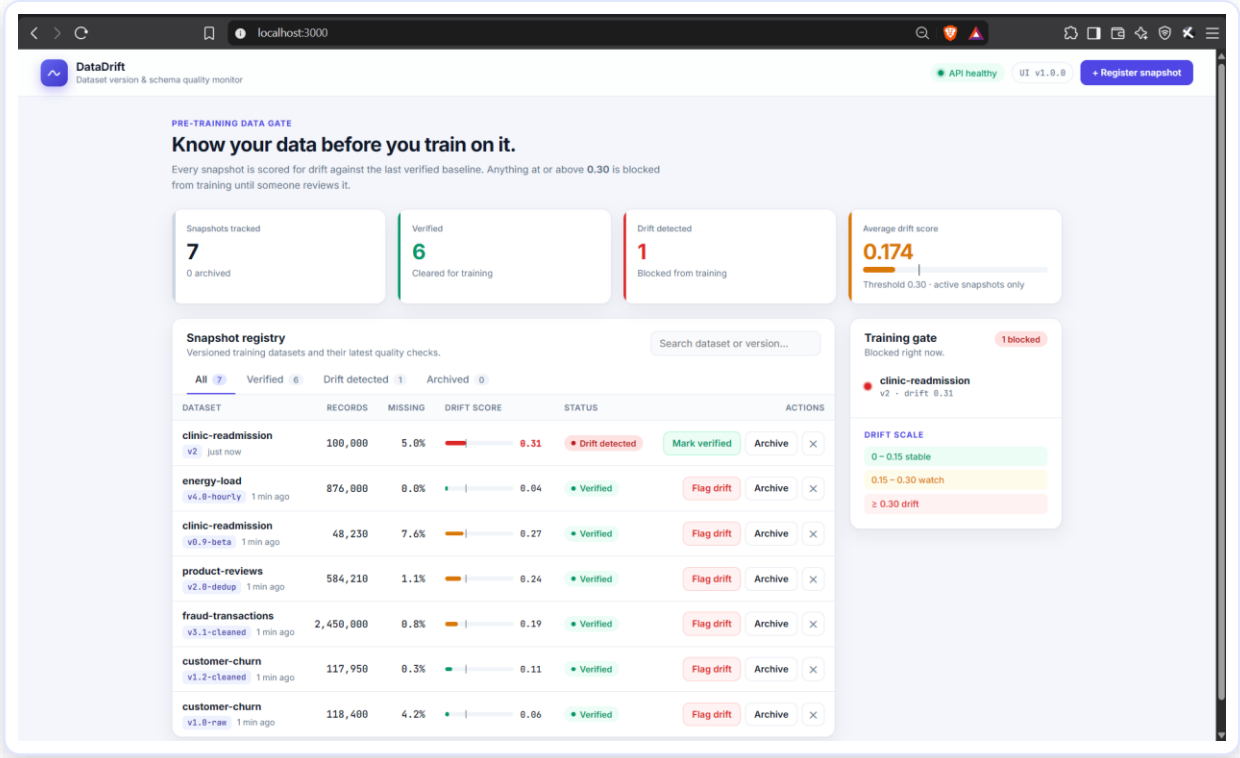
Each dataset refresh is stored with its version tag, record count, missing values and a drift score from 0 to 1.

Gate drifted data

A score of 0.30 or more marks the snapshot DRIFT_DETECTED and blocks it from training until reviewed.

Alert the team

Flag drift in the UI and a Prometheus alert fires; Grafana turns red within about 30 seconds.

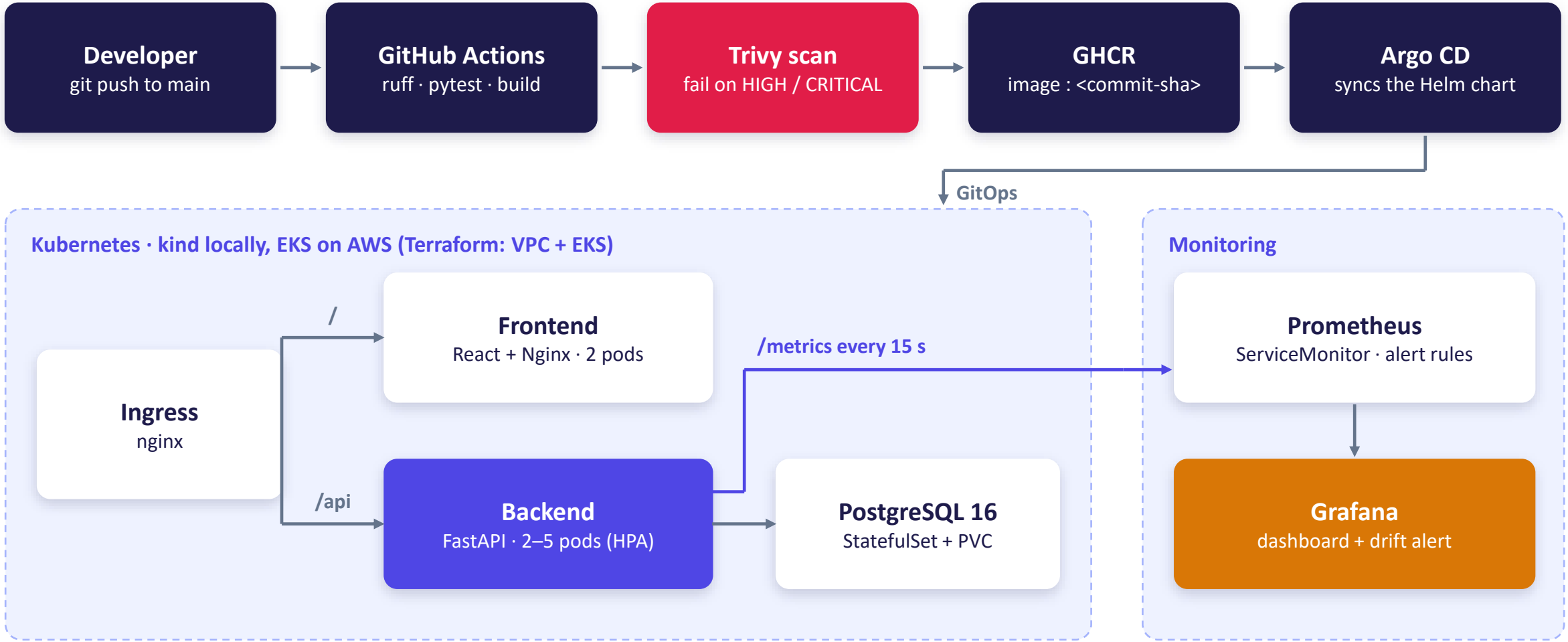


A snapshot registered at drift 0.31 is flagged and blocked automatically

0 – 0.15 stable | **0.15 – 0.30 watch** | **≥ 0.30 drift**

ARCHITECTURE

From git push to a monitored cluster



RUBRIC COVERAGE

Ten modules, one pipeline

Module	Points	What I built
M1 Application	10	FastAPI + PostgreSQL 16 + Alembic, React dashboard, GET/POST/PUT/DELETE API
M2 Testing	10	14 pytest tests on a throwaway SQLite database, ruff lint and format
M3 Git and GitHub	5	Public repository, 30+ descriptive commits, .gitignore for secrets and build output
M4 Docker	10	Non-root images, multi-stage frontend, Compose with health-checked startup
M5 CI/CD	15	GitHub Actions: test → build → scan → push to GHCR with SHA tags → GitOps tag bump
M6 DevSecOps	5	Trivy gate on HIGH/CRITICAL; 43 base-image CVEs found and fixed
M7 Terraform	15	VPC, subnets, NAT, IAM, EKS cluster and node group: 25 resources
M8 Kubernetes + Helm	15	Helm chart deployed by Argo CD (GitOps): Deployments, StatefulSet, Services, Ingress, HPA
M9 Observability	10	ServiceMonitor, drift alert rule, auto-loaded Grafana dashboard
M10 Presentation	5	README, plus slides and video in demo/; live push → pipeline → Argo CD rollout

A small API with a real database

FastAPI backend

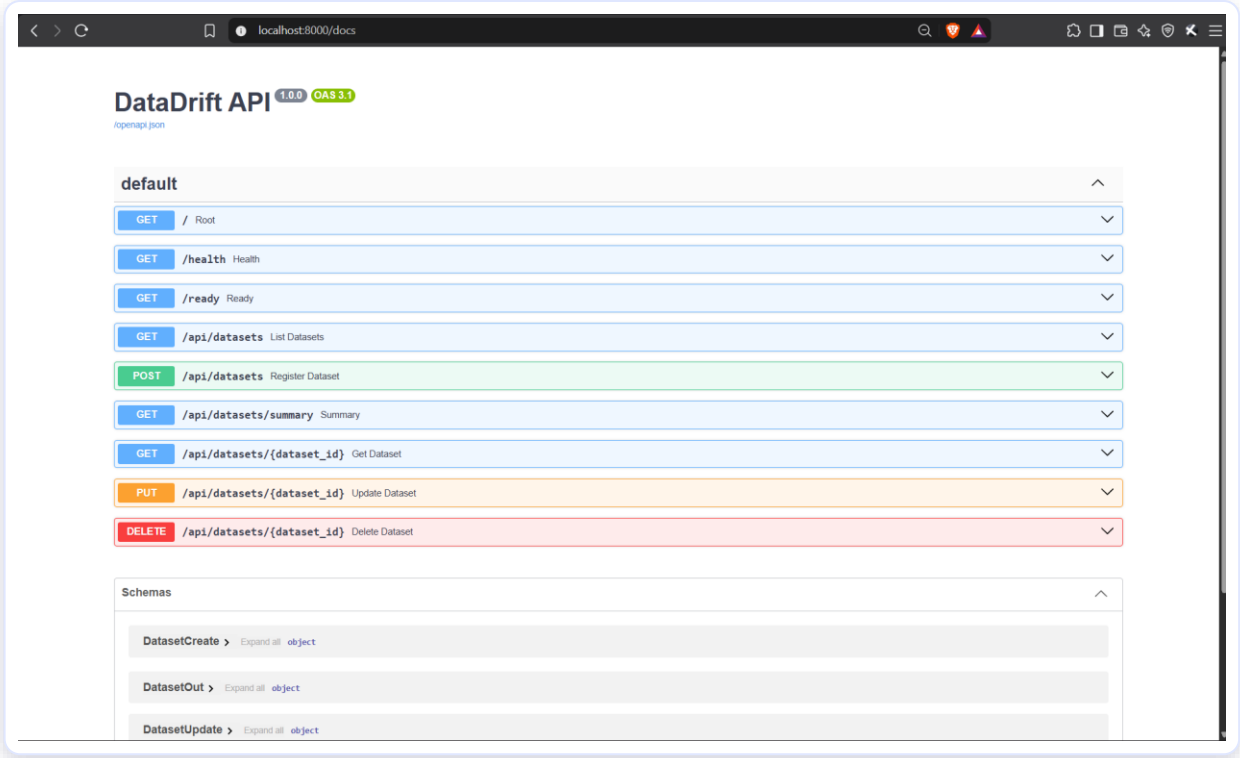
GET, POST, PUT and DELETE on /api/datasets, plus /health, /ready and /metrics. Status follows the drift score.

PostgreSQL 16 + Alembic

Migration 0001 creates the table with unique and range constraints; an advisory lock stops replicas migrating at once.

React dashboard

KPIs, a searchable registry, a register form, and one-click Flag drift / Mark verified.



Swagger UI at /docs: every endpoint, generated by FastAPI

14 tests guard every build

14 / 14

tests passing in 0.39 s

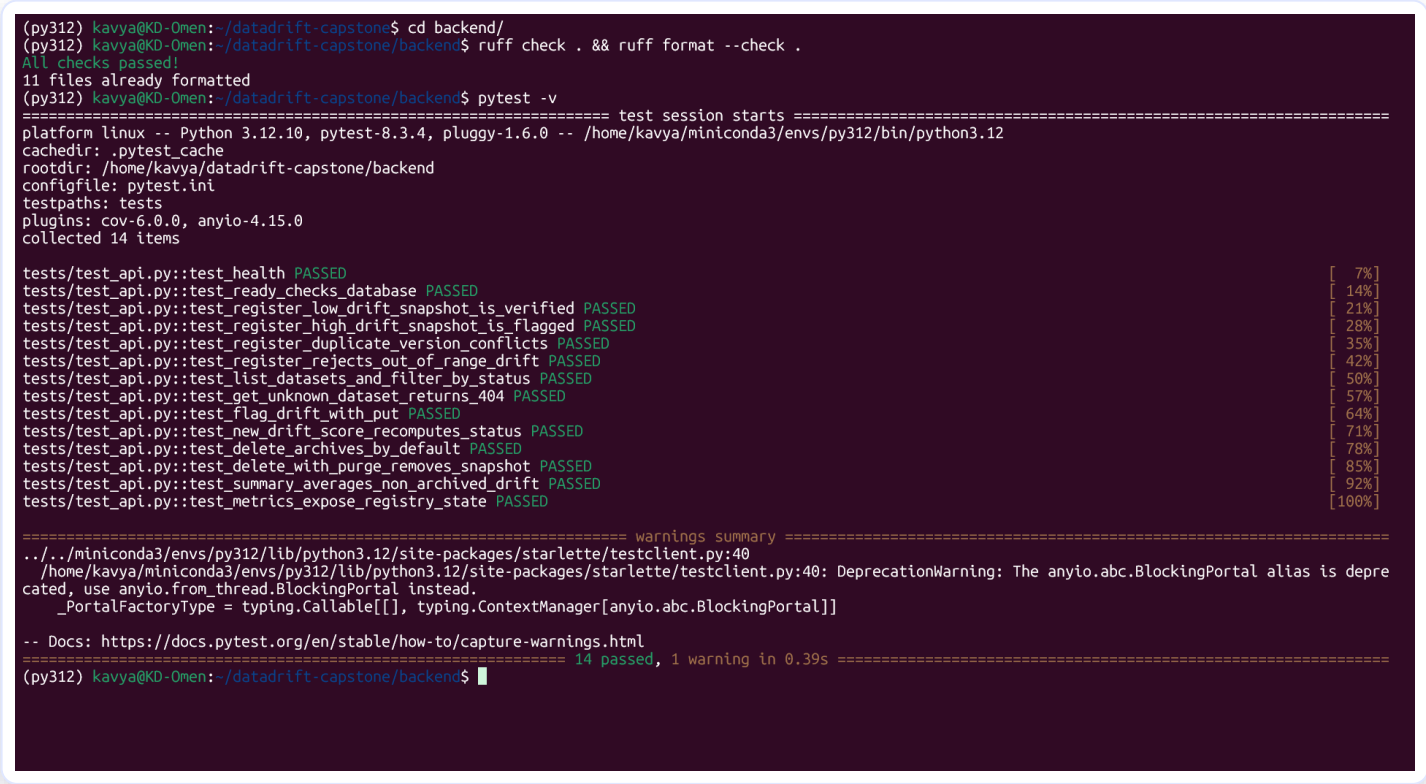
9

endpoints covered, including errors

SQLite

throwaway test database, never Postgres

ruff and pytest run first in CI: a failing test stops the pipeline before any image is built.



ruff check, ruff format --check and pytest -v

One commit per step, in the open

```
(py312) kavya@KD-Omen:~/datadrift-capstone/backend$ cd ..
(py312) kavya@KD-Omen:~/datadrift-capstone$ git log --oneline | head -25
b2ecc0a Add screenshots folder and keep local demo notes out of version control
29f5455 Document architecture, API, pipeline and deployment in README
2e44beb Add CI/CD pipeline: tests, Trivy scan, GHCR push and Helm deploy
effd96f Add troubleshooting lab with broken image and selector manifests
a082465 Add scripts to create the kind cluster, deploy with Helm and load test
4000283 Add Prometheus scraping, drift alert rules and Grafana dashboard
495ac4c Add Helm chart with backend, frontend, Postgres, ingress and HPA
ca73972 Add EBS CSI and metrics-server add-ons to the EKS cluster
0d4466b Label per-snapshot drift gauge with status for alerting
40f4963 Add Terraform for AWS VPC and EKS cluster with managed node group
308ecb4 Add seed script that registers sample snapshots via the API
5aefe1e Containerize backend and frontend as non-root images with Compose stack
c7bc8d8 Add React dashboard for dataset snapshots and drift flags
f456048 Add pytest suite on SQLite and ruff lint config
0e5b0ef Add Alembic migration for the datasets table
e04d626 Add FastAPI dataset registry API with drift metrics
6753f68 Initialize DataDrift repository with ignore rules
(py312) kavya@KD-Omen:~/datadrift-capstone$ git log --format='%h  %an  %s' | head -5
b2ecc0a  KavyaDhyani  Add screenshots folder and keep local demo notes out of version control
29f5455  KavyaDhyani  Document architecture, API, pipeline and deployment in README
2e44beb  KavyaDhyani  Add CI/CD pipeline: tests, Trivy scan, GHCR push and Helm deploy
effd96f  KavyaDhyani  Add troubleshooting lab with broken image and selector manifests
a082465  KavyaDhyani  Add scripts to create the kind cluster, deploy with Helm and load test
(py312) kavya@KD-Omen:~/datadrift-capstone$
```

git log: each commit is one logical step

Public repository

github.com/KavyaDhyani/datadrift-capstone

23 descriptive commits

Each message says what changed: no “update” or “fix”.

Secrets stay out

.gitignore covers .env, .venv, __pycache__, node_modules, kubeconfig, *.tfstate and *.tfvars.

Small, non-root images

Backend image

python:3.12-slim, runs as uid 10001, pip and setuptools removed, built-in health check.

Multi-stage frontend

node:22-alpine builds the React app; nginx-unprivileged serves it as uid 101 on port 8080.

One-command stack

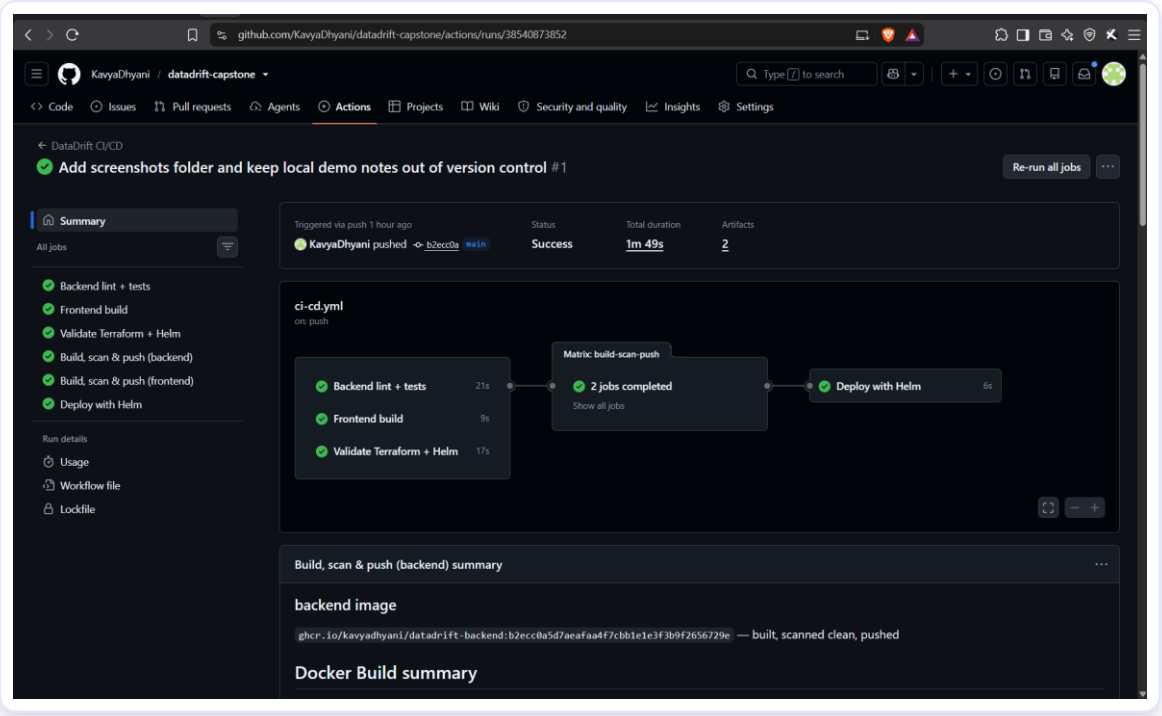
docker compose up --build starts Postgres, then the backend (after migrating), then the frontend.

```
(py312) kavya@KD-Omen:~/datadrift-capstone$ docker compose exec backend id
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
(py312) kavya@KD-Omen:~/datadrift-capstone$ docker compose exec frontend id
uid=101(nginx) gid=101(nginx) groups=101(nginx)
(py312) kavya@KD-Omen:~/datadrift-capstone$ docker images | grep datadrift
datadrift-backend:ca73972                                                e7a229ca7d5d       306MB         71.2MB
datadrift-backend:local                                                  62ede8c860f0       306MB         71.2MB   U
datadrift-frontend:ca73972                                               155fb868abb5       113MB           32MB
datadrift-frontend:local                                                 bf4076738e3d       113MB           32MB   U
(py312) kavya@KD-Omen:~/datadrift-capstone$ grep -n "FROM\|USER" backend/Dockerfile frontend/Dockerfile
backend/Dockerfile:1:FROM python:3.12-slim
backend/Dockerfile:24:USER 10001
frontend/Dockerfile:2:FROM node:22-alpine AS build
frontend/Dockerfile:10:FROM nginxinc/nginx-unprivileged:1.29-alpine
frontend/Dockerfile:12:USER root
frontend/Dockerfile:18:USER 101
(py312) kavya@KD-Omen:~/datadrift-capstone$
```

```
(py312) kavya@KD-Omen:~/datadrift-capstone$ docker compose up --build -d
[+] Building 0.5s (31/31) FINISHED
 => [internal] load local bake definitions                                                
```

Every push: test, build, scan, ship

1. **Test** — ruff + pytest
2. **Frontend** — npm build
3. **Validate** — terraform, helm
4. **Ship images** — Trivy → GHCR
5. **GitOps** — tag → Argo CD



Tagged with the commit SHA

ghcr.io/kavyadhyani/datadrift-backend:<sha>, never latest: every pod traces back to a commit.

Gates in order

Steps 1–3 run in parallel; images are built only if all pass. Pull requests build and scan but never push.

Hand-off to Argo CD

The last job commits the new SHA to values-gitops.yaml; Argo CD deploys it.

The scan found real CVEs, and the build fixed them

Frontend · nginx-unprivileged base

43 → **0**

HIGH CVEs in Alpine packages (c-ares, curl, …)

Fix: apk upgrade --no-cache in the runtime stage

Backend · python:3.12-slim

HIGH → **0**

CVEs in pip’s vendored urllib3 and in setuptools

Fix: uninstall pip, setuptools and wheel after installing dependencies



Example: CVE-2026-33630

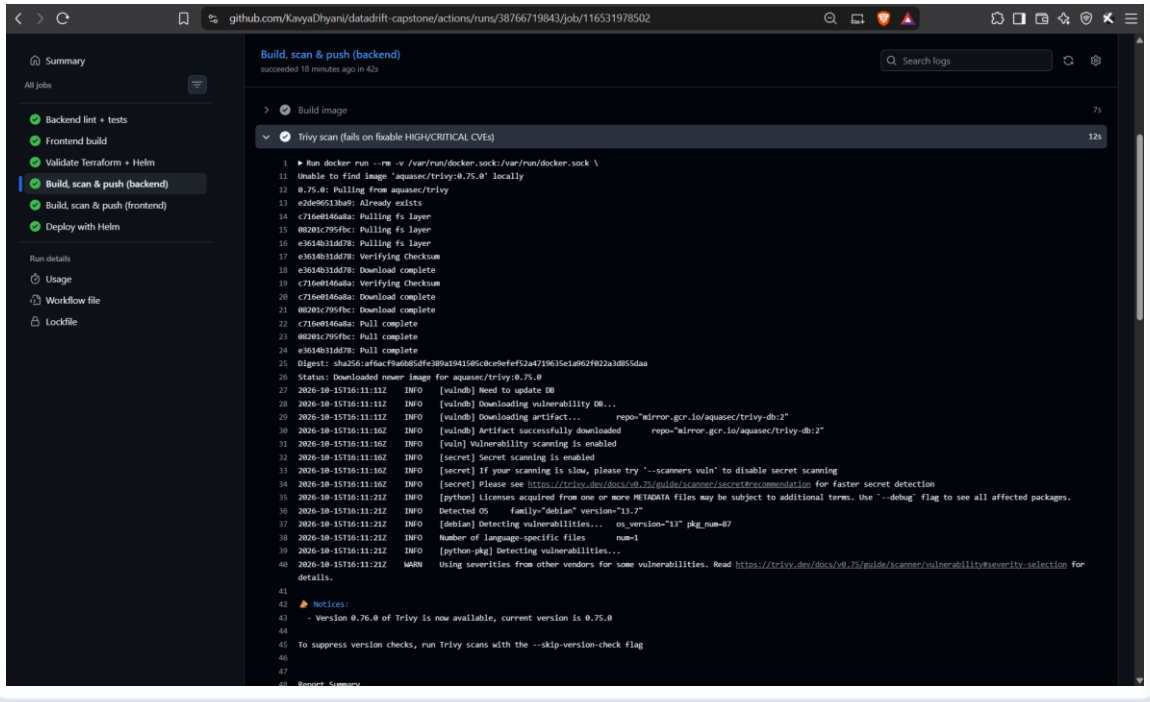
A use-after-free in c-ares, the DNS library curl uses. A fixed package existed, so upgrading removed it.



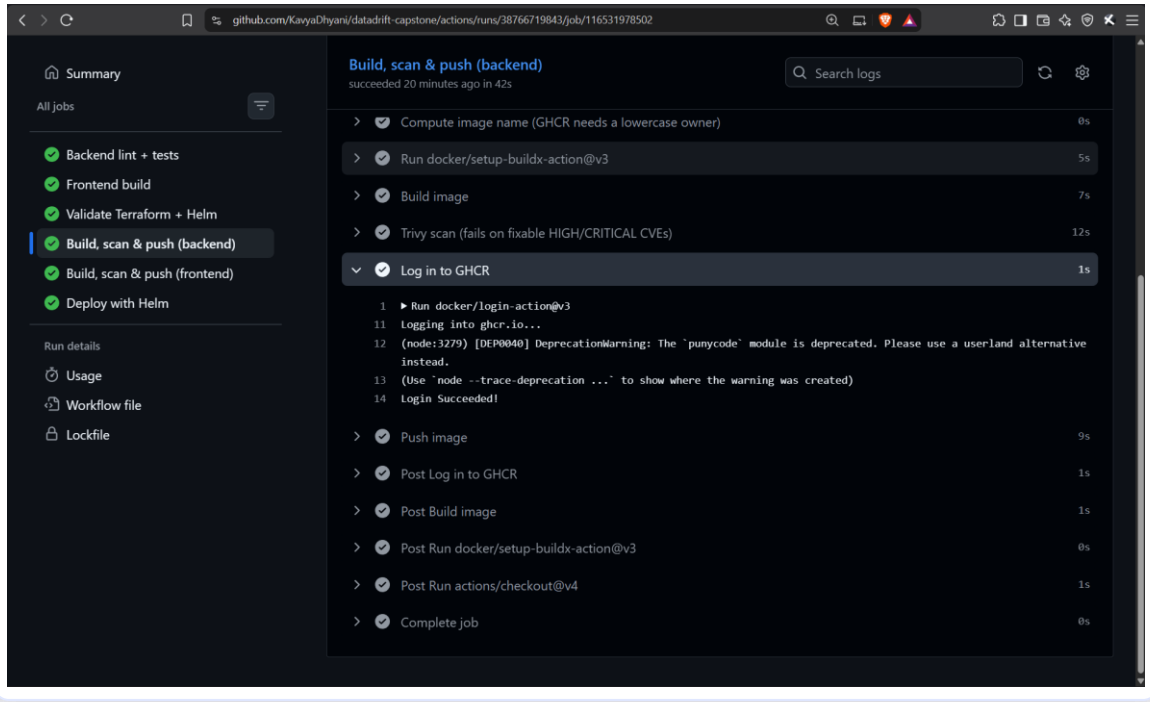
The gate in CI

--severity HIGH,CRITICAL --ignore-unfixed --exit-code 1: a fixable vulnerability fails the build.

Scanned first, pushed only if clean



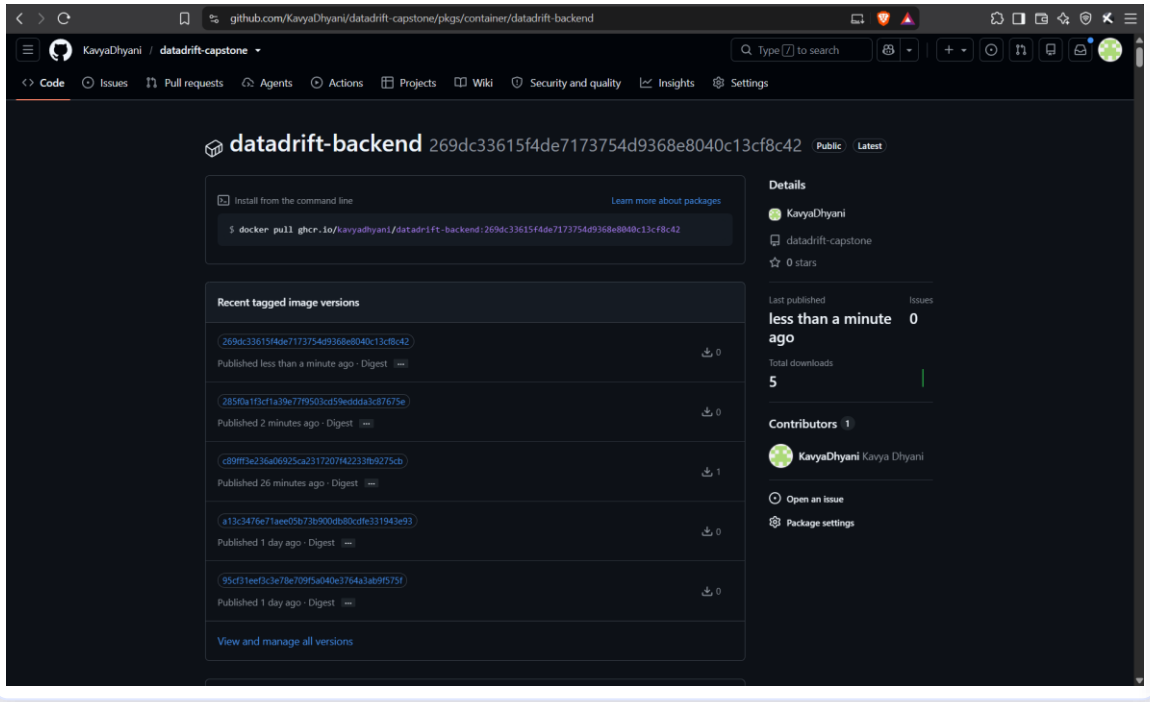
Trivy scan step: fresh vulnerability DB, OS + Python packages, secrets



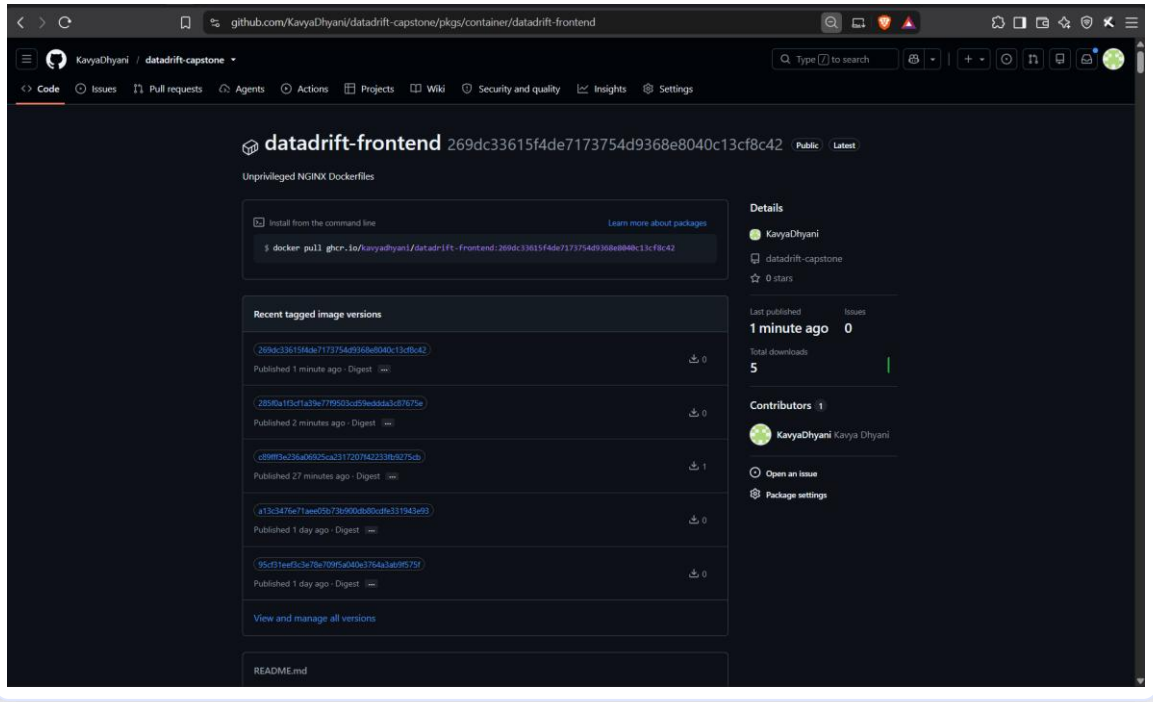
Then the job logs in to ghcr.io with GITHUB_TOKEN and pushes

The Trivy step runs with --exit-code 1 before the GHCR login: a fixable HIGH or CRITICAL CVE stops the job, so a vulnerable image is never pushed.

Every image is tagged with its commit



ghcr.io/kavyadhyani/datadrift-backend: one tag per commit SHA



ghcr.io/kavyadhyani/datadrift-frontend: the same SHAs

Both packages are public, so the cluster pulls them anonymously. A running pod’s image tag points straight back to the commit that built it, and no image is ever tagged latest.

AWS VPC and EKS, written as code

25
resources planned

2 AZs
ap-south-1a / 1b

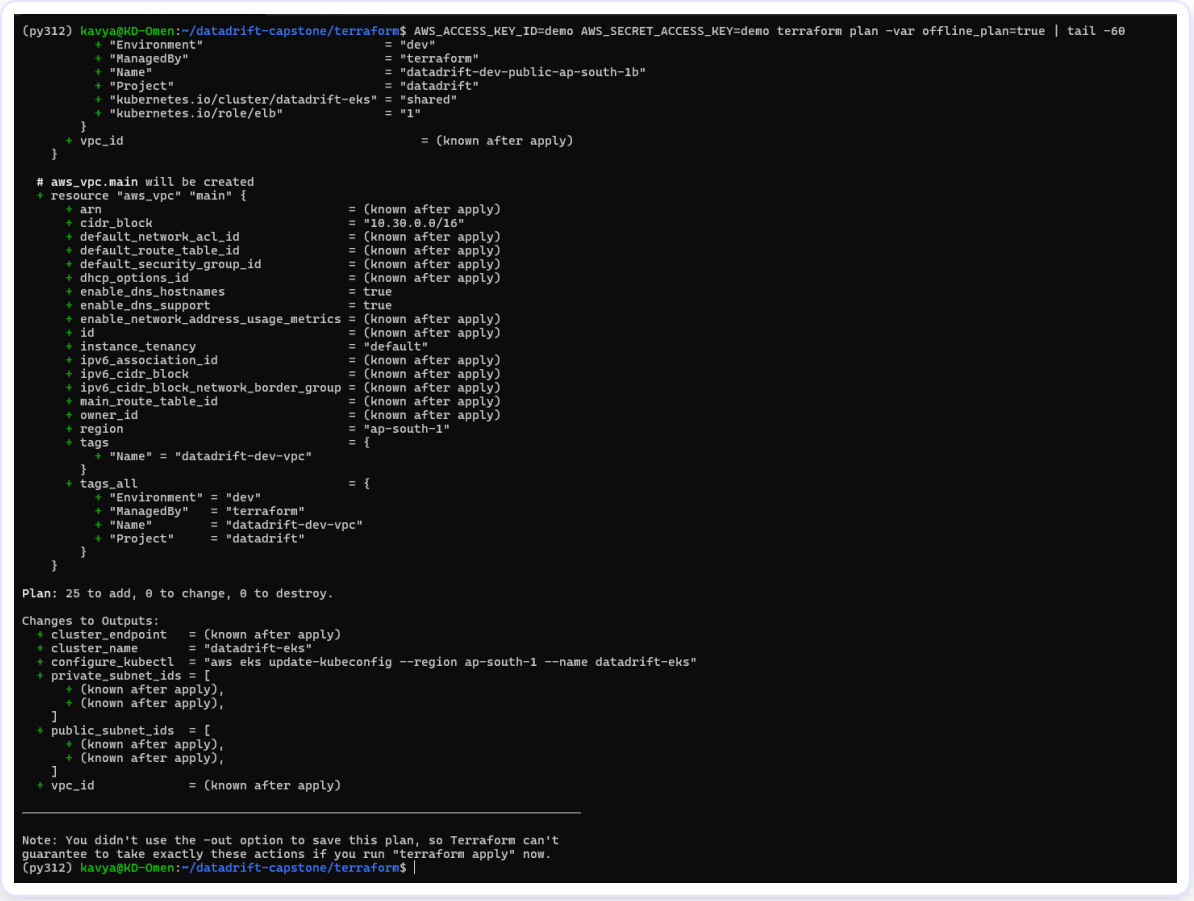
Network

VPC 10.30.0.0/16, two public and two private subnets, internet gateway, NAT gateway, route tables.

EKS

Cluster, IAM roles, managed node group of 2–4 × t3.medium in private subnets, EBS CSI and metrics-server add-ons.

Planned, not applied: no AWS account was used, so plan runs offline with dummy keys. Nothing was created, so nothing costs money; plan -destroy confirms it.



terraform plan: 25 to add, 0 to change, 0 to destroy

One Helm chart, running on kind

Deployments

backend 2–5 pods, frontend 2 pods; rolling updates with maxUnavailable 0

StatefulSet

PostgreSQL 16 with a 1 Gi persistent volume

Services + Ingress

ClusterIP services; / → frontend, /api → backend

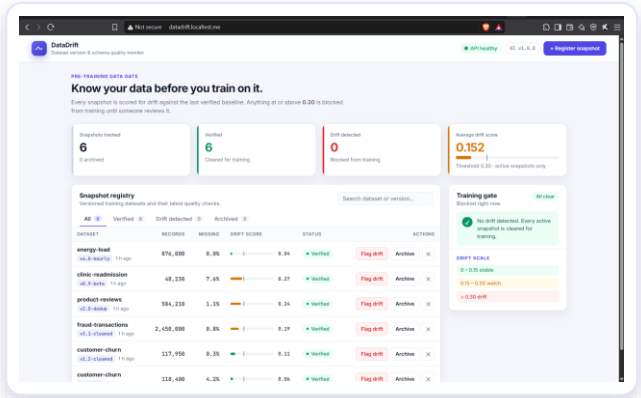
HPA

scales the backend on 60% CPU

```
(py312) kavya@KD-Omen:~/datadrift-capstone$ kubectl get pods -n datadrift -o wide
NAME                                 READY   STATUS    RESTARTS      AGE     IP            NODE                       NOMINATED NODE   READINESS GATES
datadrift-backend-5899f589d-4nnkc    1/1     Running   0             105s    10.244.0.27   datadrift-control-plane   <none>           <none>
datadrift-backend-5899f589d-bg8tn    1/1     Running   0             118s    10.244.0.26   datadrift-control-plane   <none>           <none>
datadrift-backend-5899f589d-vqc7p    1/1     Running   0             2m11s   10.244.0.25   datadrift-control-plane   <none>           <none>
datadrift-backend-5899f589d-vrq29    1/1     Running   0             2m24s   10.244.0.22   datadrift-control-plane   <none>           <none>
datadrift-frontend-6c54bdc869-5pxp5  1/1     Running   0             2m12s   10.244.0.24   datadrift-control-plane   <none>           <none>
datadrift-frontend-6c54bdc869-88jlc  1/1     Running   0             2m24s   10.244.0.23   datadrift-control-plane   <none>           <none>
datadrift-postgres-0                 1/1     Running   1 (39m ago)   97m     10.244.0.15   datadrift-control-plane   <none>           <none>
(py312) kavya@KD-Omen:~/datadrift-capstone$ kubectl get svc -n datadrift
NAME                TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
datadrift-backend   ClusterIP   10.96.252.40   <none>        8000/TCP   97m
datadrift-frontend  ClusterIP   10.96.25.195   <none>        80/TCP     97m
datadrift-postgres  ClusterIP   10.96.170.35   <none>        5432/TCP   97m
(py312) kavya@KD-Omen:~/datadrift-capstone$ kubectl get ingress,hpa -n datadrift
NAME                                                CLASS   HOSTS                   ADDRESS     PORTS   AGE
ingress.networking.k8s.io/datadrift                 nginx   datadrift.localtest.me  localhost   80      97m

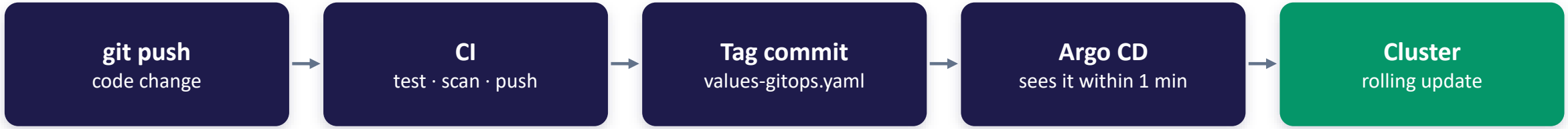
NAME                                                                REFERENCE                          TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
horizontalpodautoscaler.autoscaling/datadrift-backend   Deployment/datadrift-backend   cpu: 2%/60%   2         5         4          97m
(py312) kavya@KD-Omen:~/datadrift-capstone$ helm list -n datadrift
NAME        NAMESPACE   REVISION   UPDATED                                   STATUS     CHART              APP VERSION
datadrift   datadrift   5          2026-10-14 10:24:16.879353494 +0000 UTC   deployed   datadrift-1.0.0    1.0.0
(py312) kavya@KD-Omen:~/datadrift-capstone$
```

kubectl get pods, svc, ingress, hpa and helm list



The app through the Ingress

Git decides what runs in the cluster



Application = chart + values

argocd/datadrift-application.yaml points Argo CD at helm/datadrift and values-gitops.yaml.

Automated, self-healing sync

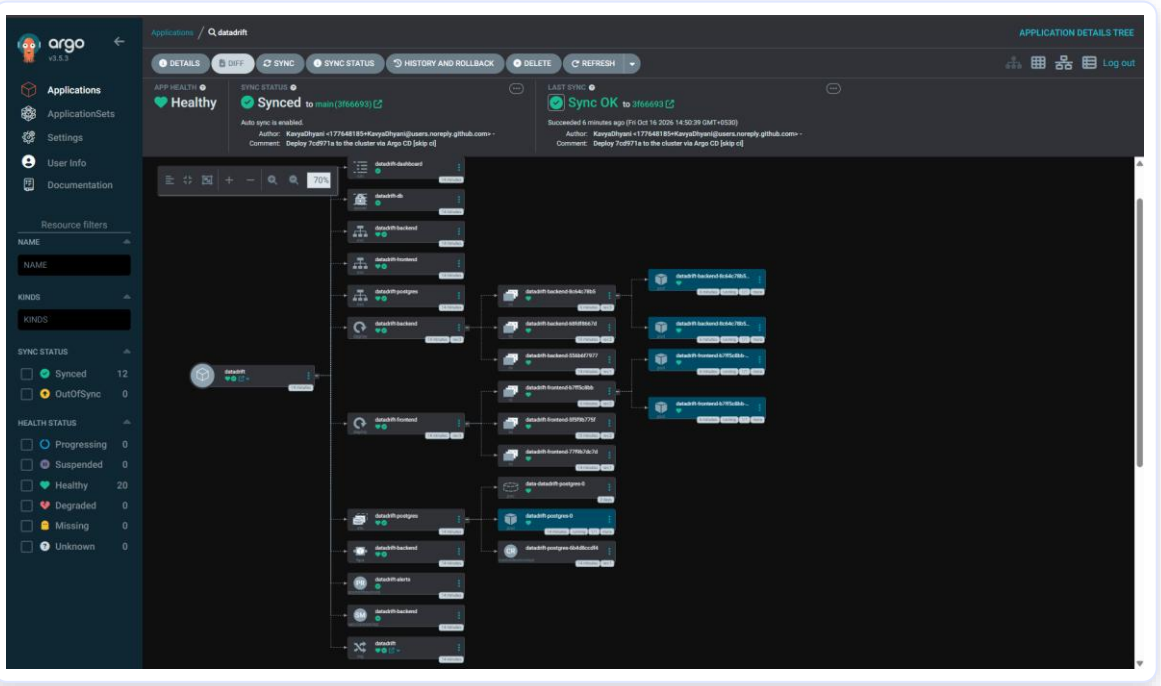
prune removes objects deleted from Git; selfHeal undoes manual kubectl changes.

Rollback is a git revert

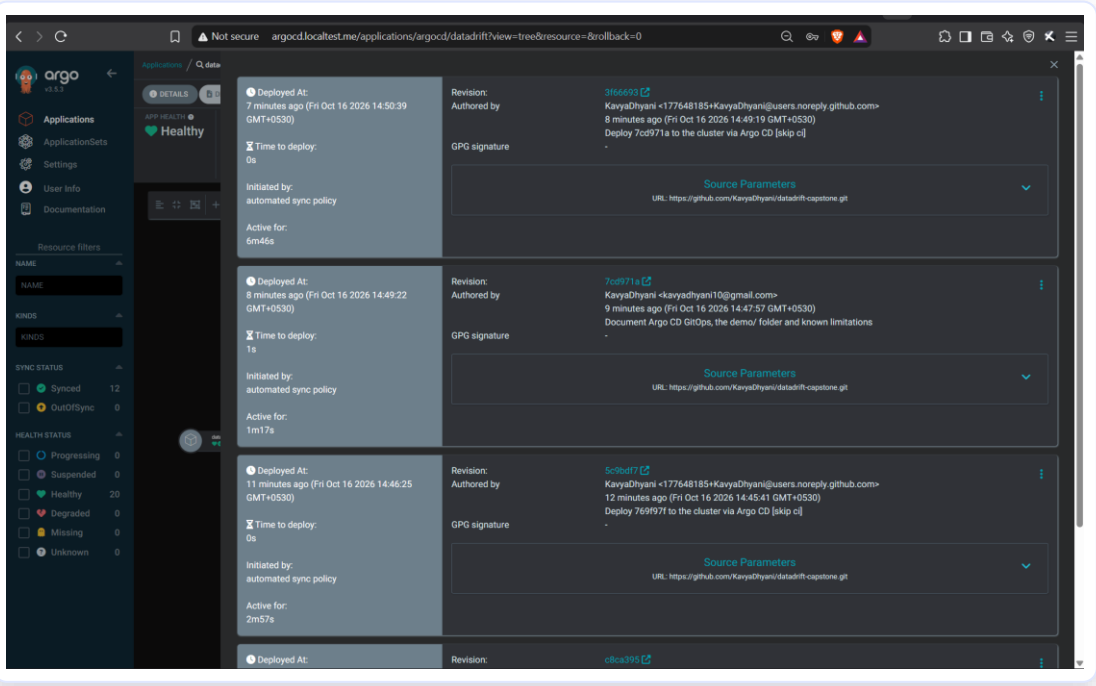
Reverting the tag commit puts the previous image back. Every release is in Git history.

```
# helm/datadrift/values-gitops.yaml
backend:
  image:
    repository: ghcr.io/kavyadhyani/
                datadrift-backend
    tag: 769f97f…    # written by CI
frontend:
  image:
    tag: 769f97f…    # same commit
```

Argo CD keeps the cluster equal to Git



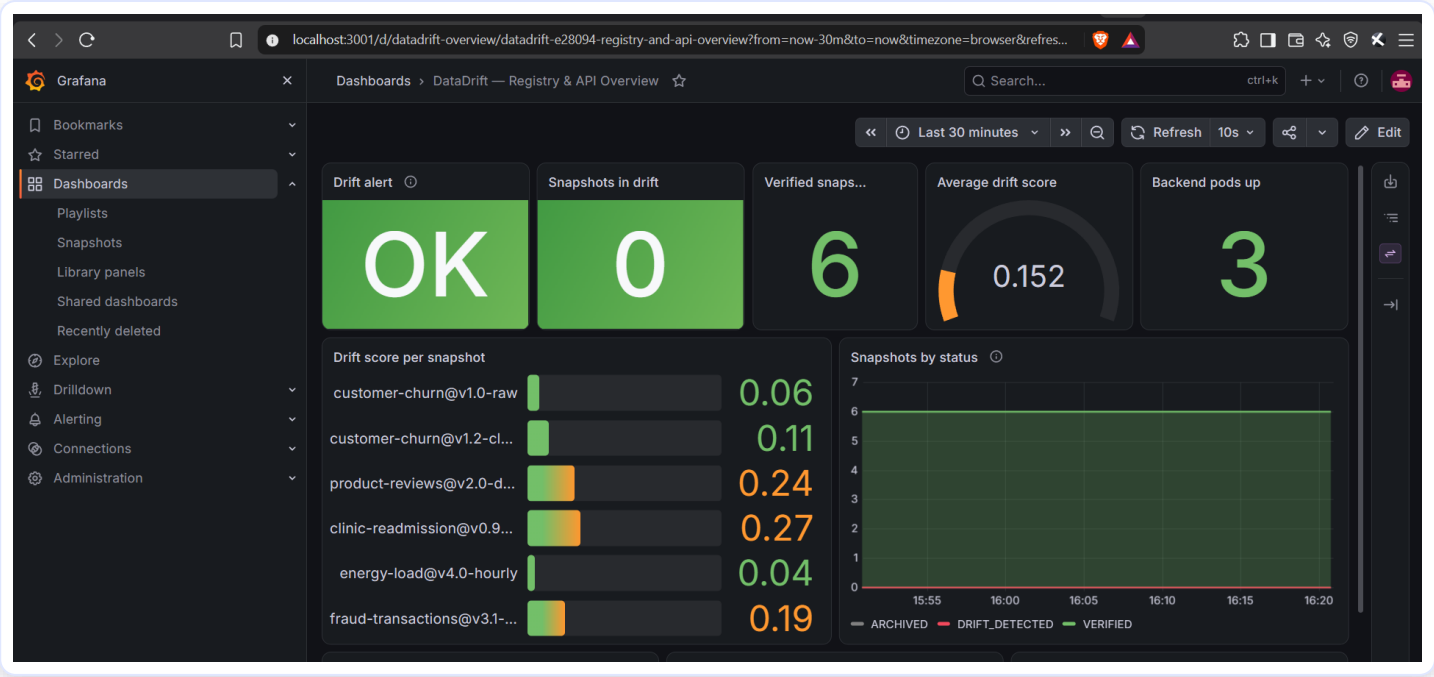
Resource tree: every object from the chart, Synced and Healthy



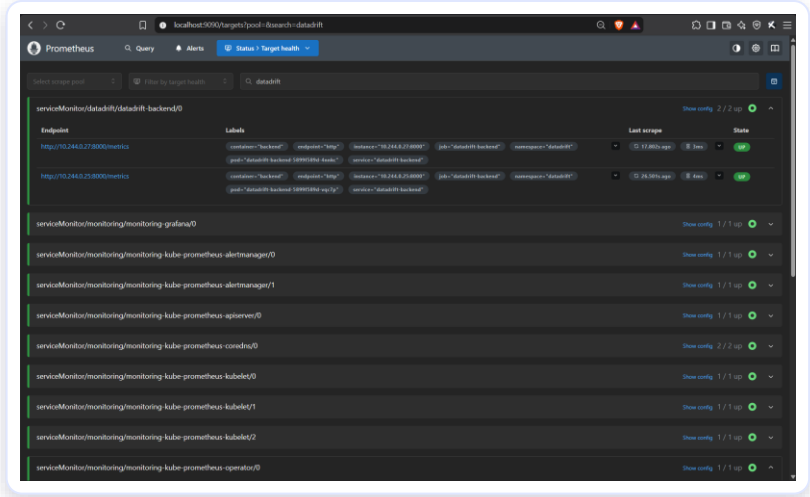
History: one automated sync per push, each a Git commit

CI never touches the cluster: it commits the new tag, and Argo CD (auto-sync, prune, self-heal) pulls the change and rolls it out within a minute.

Metrics in, dashboard out



Grafana: DataDrift — Registry & API Overview, loaded by the Helm chart



Prometheus targets: 2/2 backend pods UP

/metrics

requests, latency, and the drift score of every snapshot

ServiceMonitor

Prometheus scrapes each backend pod every 15 s

12 panels

drift, status, traffic, errors, CPU, replicas

THE HIGHLIGHT

Flag drift, and the alert fires in about 30 seconds

Flag drift
PUT /api/datasets/{id}

→

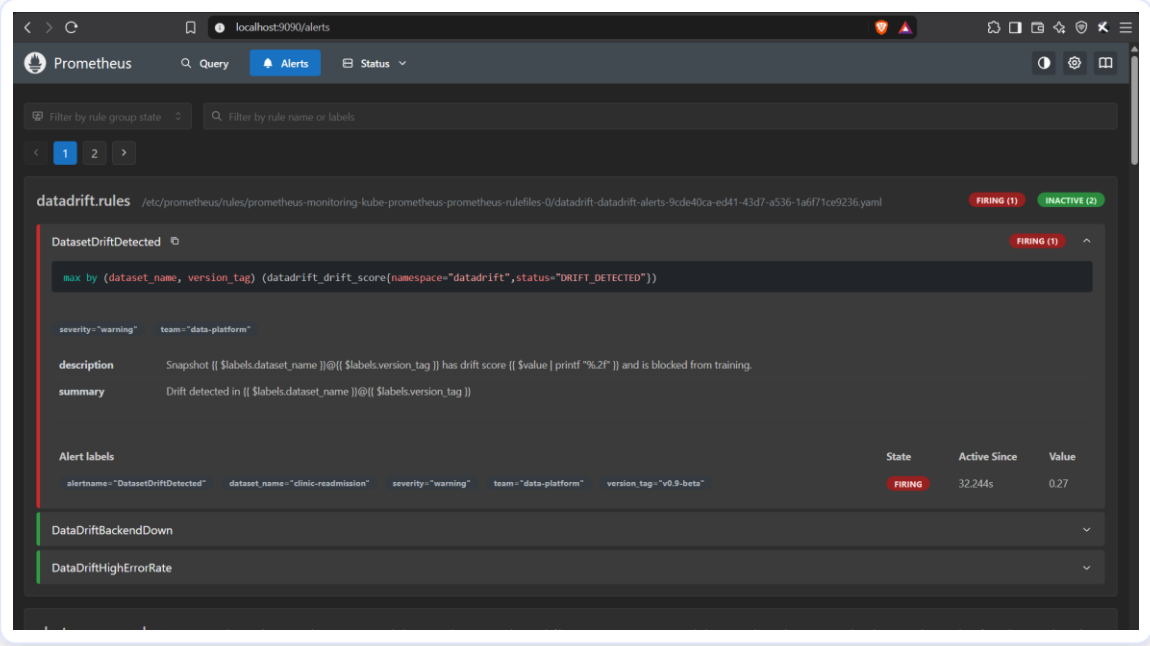
/metrics
status = DRIFT_DETECTED

→

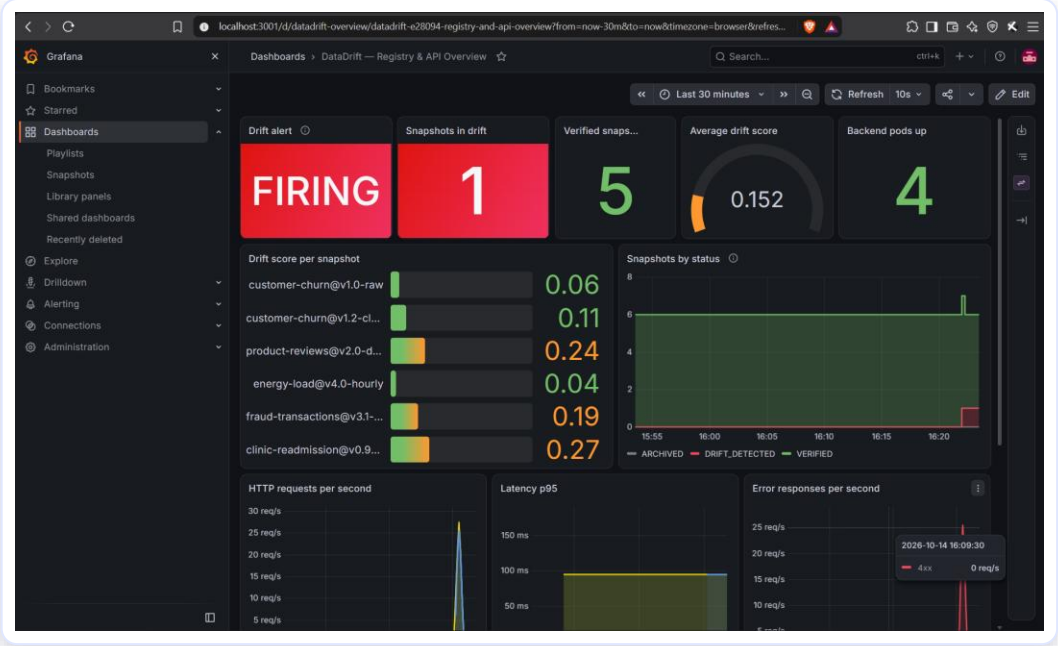
PrometheusRule
DatasetDriftDetected

→

Grafana
Drift alert: FIRING



Prometheus: firing, labelled clinic-readmission@v0.9-beta



Grafana: the Drift alert panel turns red

The backend scales out under load

2 → 4
backend pods under load

60%
CPU target per pod

5 min
before scaling back down

```
(py312) kavya@KD-Omen:~/datadrift-capstone$ kubectl get hpa -n datadrift -w
NAME                 REFERENCE                       TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
datadrift-backend    Deployment/datadrift-backend    cpu: 2%/60%   2         5         2          127m
datadrift-backend    Deployment/datadrift-backend    cpu: 45%/60%   2         5         2          128m
datadrift-backend    Deployment/datadrift-backend    cpu: 98%/60%   2         5         2          128m
datadrift-backend    Deployment/datadrift-backend    cpu: 95%/60%   2         5         4          128m
datadrift-backend    Deployment/datadrift-backend    cpu: 78%/60%   2         5         4          128m
datadrift-backend    Deployment/datadrift-backend    cpu: 60%/60%   2         5         4          129m
datadrift-backend    Deployment/datadrift-backend    cpu: 52%/60%   2         5         4          129m
datadrift-backend    Deployment/datadrift-backend    cpu: 53%/60%   2         5         4          129m
datadrift-backend    Deployment/datadrift-backend    cpu: 52%/60%   2         5         4          130m
datadrift-backend    Deployment/datadrift-backend    cpu: 53%/60%   2         5         4          130m
datadrift-backend    Deployment/datadrift-backend    cpu: 37%/60%   2         5         4          131m
datadrift-backend    Deployment/datadrift-backend    cpu: 3%/60%    2         5         4          131m
datadrift-backend    Deployment/datadrift-backend    cpu: 2%/60%    2         5         4          131m
datadrift-backend    Deployment/datadrift-backend    cpu: 2%/60%    2         5         4          135m
datadrift-backend    Deployment/datadrift-backend    cpu: 2%/60%    2         5         3          136m
datadrift-backend    Deployment/datadrift-backend    cpu: 2%/60%    2         5         2          136m
```

kubectl get hpa -w: CPU 98% → 4 pods, back to 2 after the load



Three load tests in Grafana

Commit to rollout, traced by the SHA

Edit
UI_VERSION → 1.1.0

→

git push
pipeline starts

→

CI green
image + tag commit

→

Argo CD sync
rolling update

→

Live
UI v1.1.0

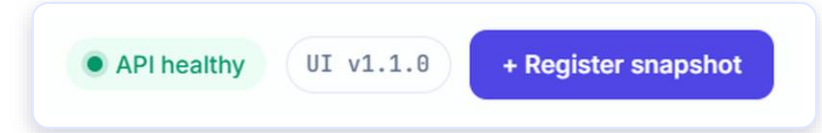
```
(py312) kavya@KD-Omen:~/datadrift-capstone$ ./scripts/deploy-local.sh --ghcr "$(git rev-parse HEAD)"
==> Deploying GHCR images ghcr.io/kavyadhyani/datadrift-*:3198bd6d93d0be923a2f6bf72f485e0533ea4817
==> helm upgrade --install datadrift
Release "datadrift" has been upgraded. Happy Helming!
NAME: datadrift
LAST DEPLOYED: Wed Oct 14 11:23:42 2026
NAMESPACE: datadrift
STATUS: deployed
REVISION: 8
DESCRIPTION: Upgrade complete
TEST SUITE: None
NOTES:
DataDrift 1.0.0 is deployed as release "datadrift" in namespace "datadrift".

  backend image : ghcr.io/kavyadhyani/datadrift-backend:3198bd6d93d0be923a2f6bf72f485e0533ea4817
  frontend image: ghcr.io/kavyadhyani/datadrift-frontend:3198bd6d93d0be923a2f6bf72f485e0533ea4817

Check it:
  kubectl get pods,svc,ingress,hpa -n datadrift

Open: http://datadrift.localtest.me
NAME                                 READY   STATUS        RESTARTS      AGE
datadrift-backend-7f9f458c4b-m52hr   1/1     Running       0             23s
datadrift-backend-7f9f458c4b-n5dz8   1/1     Running       0             2m6s
datadrift-frontend-5b5d54cfc5-c9wng  1/1     Running       0             2m6s
datadrift-frontend-5b5d54cfc5-rkwgf  1/1     Running       0             13s
datadrift-frontend-694645c658-x76df  1/1     Terminating   0             4m39s
datadrift-postgres-0                 1/1     Running       1 (96m ago)   154m

Open http://datadrift.localtest.me
(py312) kavya@KD-Omen:~/datadrift-capstone$ kubectl rollout status deploy/datadrift-frontend -n datadrift
deployment "datadrift-frontend" successfully rolled out
(py312) kavya@KD-Omen:~/datadrift-capstone$ kubectl get deploy -n datadrift -o custom-columns=NAME:.metadata.name,IMAGE:.spec.template.spec.containers[0].image
NAME               IMAGE
datadrift-backend    ghcr.io/kavyadhyani/datadrift-backend:3198bd6d93d0be923a2f6bf72f485e0533ea4817
datadrift-frontend   ghcr.io/kavyadhyani/datadrift-frontend:3198bd6d93d0be923a2f6bf72f485e0533ea4817
(py312) kavya@KD-Omen:~/datadrift-capstone$ helm history datadrift -n datadrift
REVISION        UPDATED                         STATUS          CHART             APP VERSION     DESCRIPTION
1               Wed Oct 14 08:49:06 2026        superseded      datadrift-1.0.0 1.0.0           Release "datadrift" failed: resource Deployment/datadrift/datadrift-backend not ready. statu
s: InProgress, message: Available: ...
2               Wed Oct 14 08:54:24 2026        superseded      datadrift-1.0.0 1.0.0           Upgrade complete
3               Wed Oct 14 08:59:23 2026        superseded      datadrift-1.0.0 1.0.0           Upgrade complete
4               Wed Oct 14 09:22:28 2026        superseded      datadrift-1.0.0 1.0.0           Upgrade complete
5               Wed Oct 14 10:24:16 2026        superseded      datadrift-1.0.0 1.0.0           Upgrade complete
6               Wed Oct 14 11:19:06 2026        superseded      datadrift-1.0.0 1.0.0           Upgrade complete
7               Wed Oct 14 11:21:56 2026        failed          datadrift-1.0.0 1.0.0           Upgrade "datadrift" failed: context canceled
8               Wed Oct 14 11:23:42 2026        deployed        datadrift-1.0.0 1.0.0           Upgrade complete
(py312) kavya@KD-Omen:~/datadrift-capstone$
```



The header after the rollout (zoomed)

Both Deployments run the image CI built for that commit, tagged with its SHA.

Argo CD rolls it out with maxUnavailable: 0, so the app stays up; reverting the tag commit in Git rolls it back.

CHALLENGES

What broke, and what I learned

Vulnerable base images

Trivy found 43 HIGH CVEs in a fresh nginx image. Patching in the Dockerfile and removing unused tools brought both images to zero.

Two replicas, one migration

Both backend pods ran Alembic at startup and raced. A Postgres advisory lock lets only one migrate.

HPA that wouldn’t scale down

At 52% CPU it kept 4 pods: the replica formula still rounds to 4, and scale-down waits 5 minutes by design.

Infrastructure without an AWS bill

An offline_plan variable lets terraform plan run with dummy keys, and CI deploys only when a cluster secret exists.

Thank you